

Otulia Web: Deep Dive - Chapter 3: Backend Services & API (Node.js & Express)

3.1 Introduction to the API Layer

The Otulia API is a modular, high-performance RESTful service built with Express.js. It acts as the orchestration layer between the React frontend and the MongoDB database. This chapter provides a line-by-line breakdown of the core services, middleware, and business logic that power the platform.

3.2 Security & Middleware

Before any data is served, it must pass through our security layer.

3.2.1 Authentication Middleware (`auth.middleware.js`)

This function protects private routes by verifying the user's identity.

```
const authMiddleware = async (req, res, next) => {
  // 1. Extract the token from the 'Authorization' header
  const authHeader = req.headers.authorization;
  if (!authHeader || !authHeader.startsWith("Bearer ")) {
    return res.status(401).json({ error: "NO_TOKEN" });
  }

  const token = authHeader.split(" ")[1];

  try {
    // 2. Verify the JWT using our secret key
    const decoded = jwt.verify(token, process.env.JWT_SECRET);

    // 3. Fetch the full user from DB to ensure session is still valid
    const user = await User.findById(decoded.id);
    if (!user) return res.status(401).json({ error: "USER_NOT_FOUND" });

    // 4. Attach user object to the request for use in later routes
    req.user = user;
    next(); // Proceed to the actual route handler
  } catch (err) {
    return res.status(401).json({ error: "TOKEN_INVALID_OR_EXPIRED" });
  }
};
```

3.3 The Asset Retrieval Service (`assets.routes.js`)

Retrieving luxury assets requires complex filtering, full-text search, and pagination.

3.3.1 Dynamic Filtering & Search Logic

The `/api/assets/cars` (and other category) routes use a flexible query builder.

```

router.get("/cars", async (req, res) => {
  const { search, page = 1, limit = 12, minPrice, maxPrice, acquisition } = req.query;
  let query = { status: 'Active' }; // Only show public listings

  // 1. Keyword Search: Searches across multiple fields using Regex
  if (search) {
    const regex = { $regex: search, $options: "i" };
    query.$or = [{ title: regex }, { description: regex }, { brand: regex }];
  }

  // 2. Price Range: Strict numeric filtering
  if (minPrice || maxPrice) {
    query.price = {};
    if (minPrice) query.price.$gte = Number(minPrice);
    if (maxPrice) query.price.$lte = Number(maxPrice);
  }

  // 3. Pagination & Execution
  const data = await CarAsset.find(query)
    .skip((page - 1) * limit) // Skip previous pages
    .limit(Number(limit)) // Only fetch requested amount
    .sort({ createdAt: -1 }); // Newest first

  res.json({ data, total: await CarAsset.countDocuments(query) });
});

```

3.4 Inventory Management Service (inventory.routes.js)

This service powers the Dealer Dashboard. It includes complex "Plan Gating" and real-time statistics calculation.

3.4.1 Subscription-Based Access Control

```

router.get('/dashboard', authMiddleware, async (req, res) => {
  const user = await User.findById(req.user.id).populate('myListings.item');

  // BLOCK Freemium users from advanced inventory features
  if (user.plan === 'Freemium') {
    return res.status(403).json({ error: 'INSUFFICIENT_PLAN' });
  }
}

```

3.4.2 Real-Time Stats Aggregation

The dashboard calculates business metrics on-the-fly without storing redundant data.

```

let stats = { totalAssets: listings.length, totalViews: 0 };

// Aggregating view counts from all owned items
listings.forEach(l => {
  stats.totalViews += (l.item.views || 0);
});

```

```

});

// Calculating Estimated Lead Value (Sum of prices of assets with active leads)
const leadAssetIds = await Lead.distinct('assetId', { agentId: user._id });
const estLeadValue = await CarAsset.aggregate([
  { $match: { _id: { $in: leadAssetIds } } },
  { $group: { _id: null, total: { $sum: "$price" } } }
]);

```

3.5 Lead & Communication Service (lead.routes.js)

This service manages the bridge between buyers and sellers, including email triggers.

3.5.1 The Lead Submission Flow

```

router.post("/send", authMiddleware, async (req, res) => {
  const { agentId, assetId, message, agentEmail } = req.body;

  // 1. Create persistent Lead record
  const lead = new Lead({ sender: req.user.id, agentId, assetId, message });
  await lead.save();

  // 2. Push in-app notification to Agent's user profile
  await User.findByIdAndUpdate(agentId, {
    $push: {
      notifications: { message: `New lead for asset: ${assetId}`, targetTab: "leads" }
    }
  });

  // 3. Trigger Email Notification via NodeMailer
  const mailOptions = {
    to: agentEmail,
    subject: "New Luxury Lead - Otulia",
    text: `You have a new inquiry: ${message}`
  };
  transporter.sendMail(mailOptions);
});

```

3.6 Advanced API Features

3.6.1 Global Search Suggestions

We combine Google's unofficial API with our own database records to provide high-quality search autocomplete.

```

router.get("/suggestions", async (req, res) => {
  const { q } = req.query;
  // 1. Call External API (Google)
  const googleRes = await axios.get(`https://suggestqueries.google.com/...q=${q}`);

```

```
// 2. Search Internal DB
const internalRes = await CarAsset.find({ brand: new RegExp(`^${q}`, 'i') }).limit(5);

// 3. Merge & Deduplicate
res.json([...googleRes.data[1], ...internalRes.map(c => c.brand)]);
});
```

3.6.2 The Sitemap Engine

The sitemap route generates a machine-readable XML index of the entire database.

```
router.get('/sitemap.xml', async (req, res) => {
  res.header('Content-Type', 'application/xml');
  let xml = `<?xml version="1.0" encoding="UTF-8"?><urlset ...>`;

  // Querying all models in parallel for maximum speed
  const assets = await Promise.all([CarAsset.find(), YachtAsset.find()]);

  assets.flat().forEach(asset => {
    xml += `<url><loc>${BASE_URL}/asset/${asset._id}</loc></url>`;
  });

  res.send(xml);
});
```

3.7 Error Handling & Best Practices

- **Standardized Responses:** Every error returns a consistent JSON object: `{ error: "CODE", details: "Message" }`.
- **Status Codes:**
 - 200/201 : Success
 - 401 : Auth required
 - 403 : Forbidden (Plan limit)
 - 404 : Not found
 - 500 : Server crash
- **Statelessness:** The API does not use server-side sessions. All state is carried in the JWT, allowing the API to scale horizontally across multiple servers if needed.